Registers, clocks, and the pin

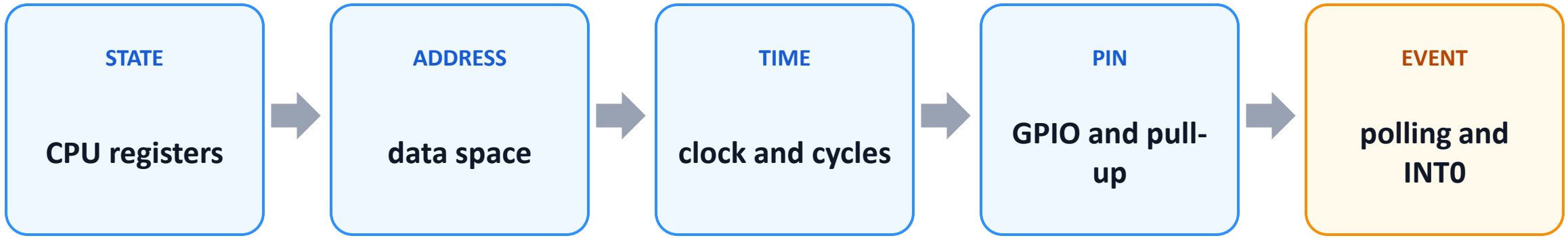
ATmega328P CPU state, timing, GPIO, polling, and an INT0 bridge



STATE · TIME · HARDWARE

SESSION 2

Session 2 roadmap



The final destination is Lab Session 2 readiness, not a complete interrupt lecture.

Session 1 system map, Session 2 focus

Session 1

CPU

executes instructions

MEMORY

stores program and data

PERIPHERALS

connect to the outside world

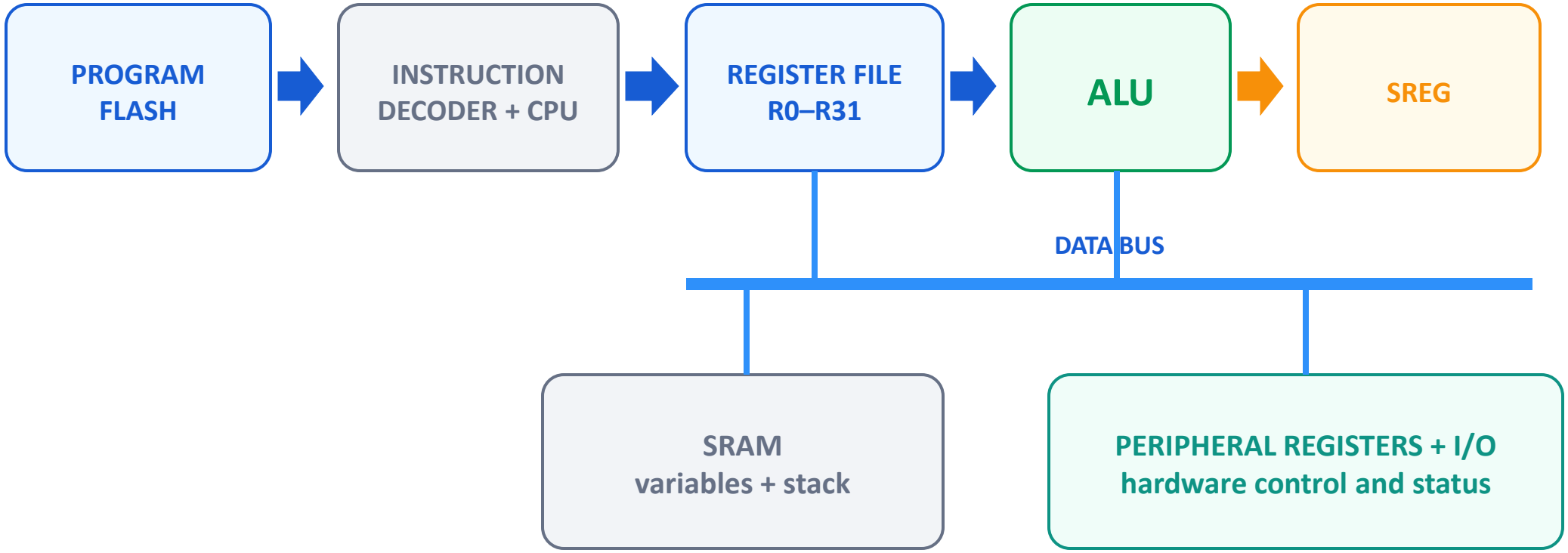
Session 2

Registers hold the CPU's immediate state.

The clock controls when that state advances.

Later, peripheral registers connect that state and timing to a physical pin.

Simplified ATmega328P CPU and data path



Instructions come from Flash. Operands move through R0–R31 and the ALU. SREG records conditions.

Registers hold the values the CPU needs now

```
result = sensor_value + offset;
```

OPERANDS sensor_value and offset

RESULT the value just calculated

NEXT which instruction comes next

CONDITION zero, carry, negative, interrupt state

The 32 × 8-bit register file

One-cycle access keeps operands close to the ALU

R0	R1	R2	R3	R4	R5	R6	R7
R8	R9	R10	R11	R12	R13	R14	R15
R16	R17	R18	R19	R20	R21	R22	R23
R24	R25	R26	R27	R28	R29	R30	R31

R16–R31 support immediate operands

R26–R31 also form X, Y, and Z

In C, the compiler normally allocates these working registers.

Working registers, I/O registers, and variables

Item	Where it lives	Who manages it	What it means
R0–R31	CPU register file	compiler or assembly	immediate operands and results
PORTB, EICRA	I/O data space	your register writes	hardware control and status
uint8_t count	SRAM or a working register	compiler	program-defined data

Peripheral-register macros are volatile hardware addresses with side effects.

The compiler may move an ordinary variable without changing its meaning.

X, Y, and Z pointer register pairs



The pair is wider than the 8-bit data path because addresses must reach more than 256 bytes.

PC, Stack Pointer, and SREG

PC

Where execution continues

14-bit word address on ATmega328P

SP

Where the stack currently sits

16-bit address; stack grows toward lower SRAM addresses

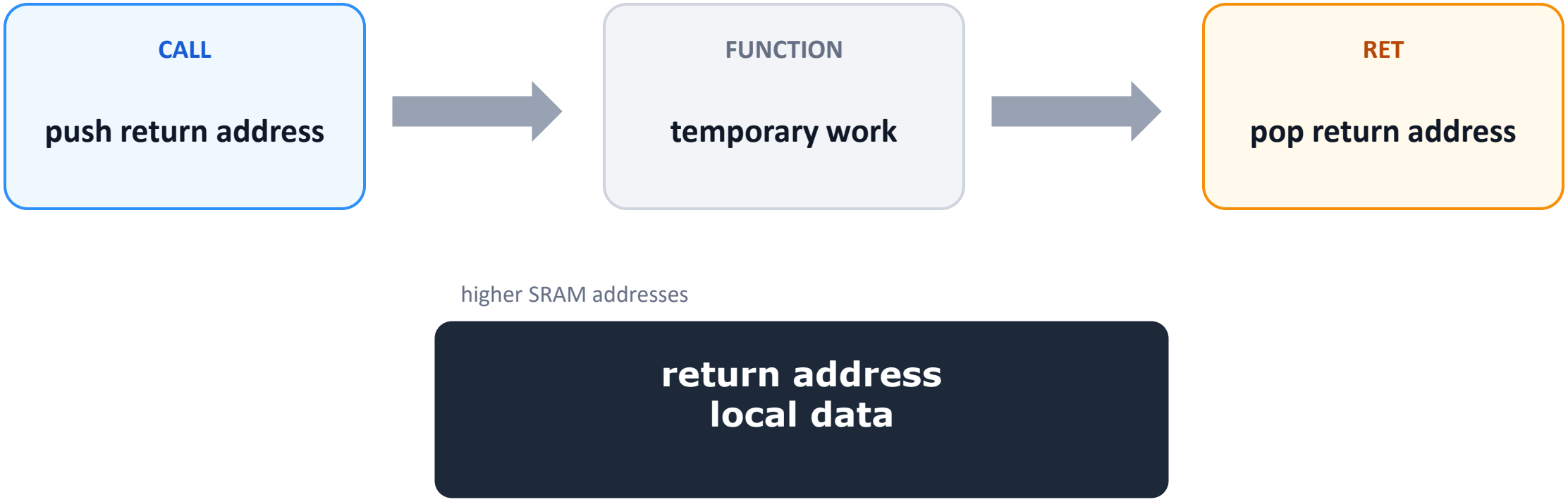
SREG

What the last operation implies

8 status bits, including global interrupt enable

Calls and interrupts need all three: a return address, stack space, and status control.

A function call uses the stack



SREG is one byte of CPU conditions



I	T	H	S	V	N	Z	C
global IRQ	bit copy	half carry	**signed result**	**overflow**	**negative**	**zero**	**carry**

Focus today: I, Z, C, N, V, and S

H supports half-carry cases such as BCD arithmetic. T supports bit-copy instructions. Both can wait.

What 8-bit actually describes

8-bit core data path

ALU and general-purpose registers

14-bit	Program Counter	**16-bit**	X, Y, Z and Stack Pointer
32 KB	Flash program memory	**16 or 32 bit**	instruction encoding

Why the range ends at 0x001F

End = Start + Size – 1

Start		Size			
0x0000	+	0x0020	– 1 =	0x001F

32 registers × 1 byte = 32 bytes = 0x20 bytes

The subtraction avoids counting one extra address.

ATmega328P data address space

Data address	Size	Region	Examples
0x0000–0x001F	32 B	working registers	R0–R31
0x0020–0x005F	64 B	standard I/O	PINx, DDRx, PORTx, SREG
0x0060–0x00FF	160 B	extended I/O	CLKPR, EICRA, timer registers
0x0100–0x08FF	2048 B	internal SRAM	variables, stack

Flash uses a separate program address space.

I/O address and unified data address

For standard I/O registers

data address = I/O address + 0x20

Register	I/O address	Data address	Why both appear
PORTB	0x05	0x25	IN/OUT view and data-space view
SREG	0x3F	0x5F	same physical register
EICRA	—	0x69	extended I/O uses data-space access

Never add 0x20 twice. First identify which address column the datasheet uses.

Address-range challenge

End = Start + Size – 1

A · EXTENDED I/O

Start 0x0060
Size 0x00A0

End = ?

B · INTERNAL SRAM

Start 0x0100
Size 0x0800

End = ?

Answers: A = 0x00FF · B = 0x08FF

Memory-mapped I/O connects software to hardware



The address is a control point, not passive storage.

The same idea later controls timers, USART, ADC, TWI, and interrupts.

The common peripheral-register pattern

Software reaches hardware through registers with distinct jobs.

CONTROL

select behavior
and enable features

STATUS

report events
and current conditions

DATA

carry values
into or out of hardware

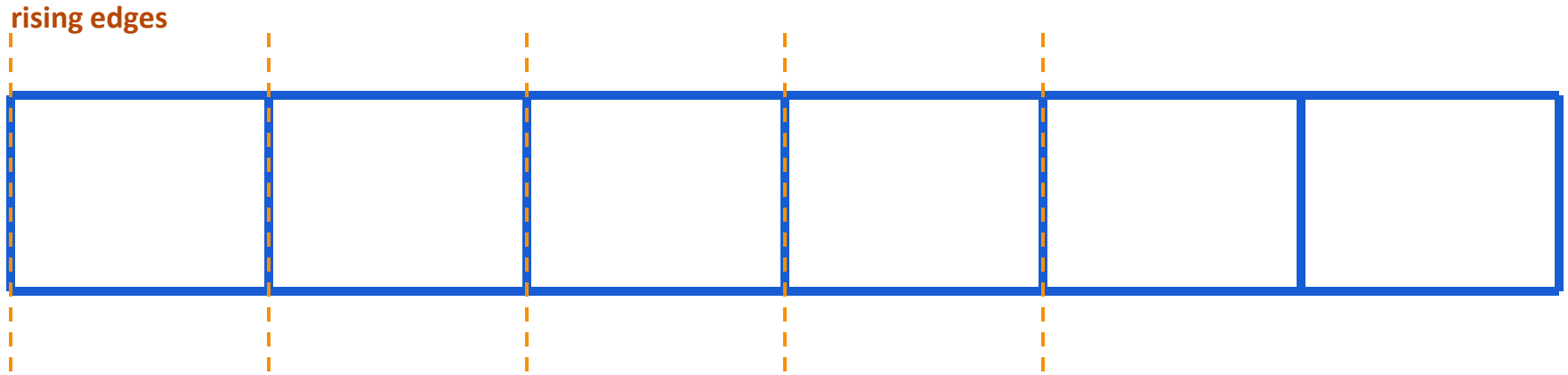
Later sessions reuse this pattern

Timer USART ADC TWI interrupt controller

Exact register names differ. The roles remain recognizable.

The clock orders CPU state changes

A synchronous CPU changes stored state at defined clock edges.





One period T spans one complete cycle. Frequency f counts cycles per second.

Frequency and period describe the same clock

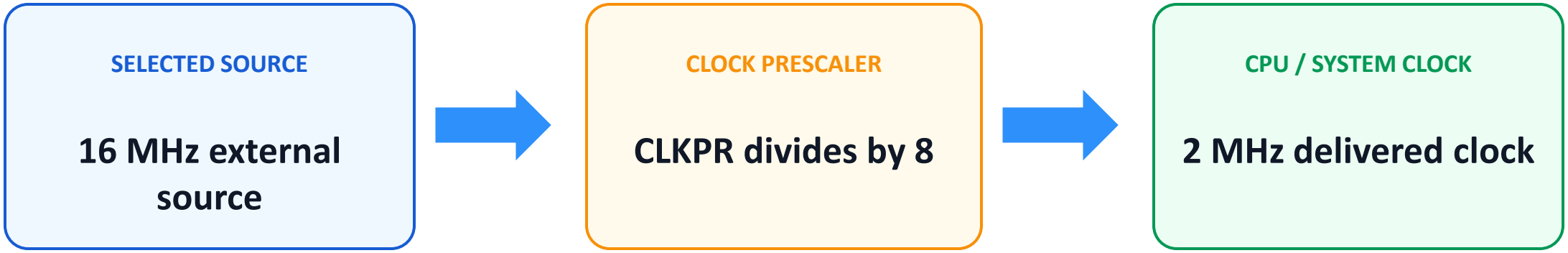
$$f = 1 / T$$

$$T = 1 / f$$

Frequency	One cycle takes
16 MHz	62.5 ns
8 MHz	125 ns
2 MHz	500 ns
1 MHz	1 µs

Clock source and CPU clock are different points

The selected source enters the chip before the runtime prescaler.



The source remains at 16 MHz. CLKPR changes the clock delivered to synchronous logic.

ATmega328P clock source options

Source	Connection	Design tradeoff
Calibrated internal RC	On-chip oscillator	Fewer parts, lower cost, compact board; lower timing accuracy
Crystal or ceramic resonator	XTAL1 and XTAL2 plus external parts	Additional components; better timing stability and accuracy
External clock	Another circuit drives XTAL1	Useful when a separate clock source already exists
Low-frequency crystal	32.768 kHz option	Low-rate or timekeeping use; special configuration

Fuse bits select the source. This session compares choices but does not program fuses.

Factory ATmega328P and Arduino Uno clock setup

BARE FACTORY DEVICE

01 Calibrated 8 MHz internal RC selected

02 CKDIV8 programmed

03 **CPU starts at 1 MHz**

ARDUINO UNO

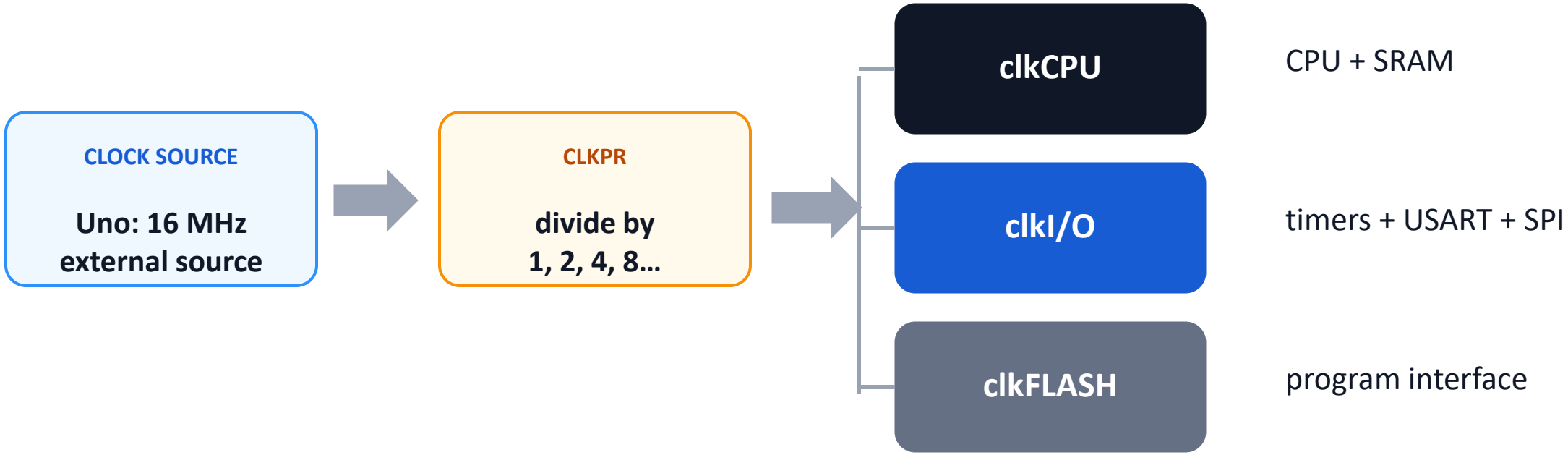
01 External 16 MHz resonator or source

02 Board fuses select that source

03 **CPU normally runs at 16 MHz**

Custom PCB decision: fewer parts with internal RC, or better timing stability with an external source.

Clock distribution inside the ATmega328P



ADC adds its own prescaler. Timer2 may use a separate 32.768 kHz clock when configured asynchronously.

CLKPR divides the system clock



CLKPR divisor	CPU frequency	Cycle period	Relative code speed
÷1	16 MHz	62.5 ns	1×
÷2	8 MHz	125 ns	1/2×
÷8	2 MHz	500 ns	1/8×

Clock cycles are not instruction counts

Many AVR arithmetic instructions

ADD · AND · EOR

often 1 cycle

Control flow and memory operations

CALL · RET · branch

can need multiple cycles

execution time = total instruction cycles / clock frequency

Use the instruction-set timing table when exact cycle counts matter.

Timing from cycle count

Example: a code path needs 1,600 cycles

CPU clock	Calculation	Elapsed time
16 MHz	1600 / 16,000,000	100 µs
2 MHz	1600 / 2,000,000	800 µs

Same instructions. Same cycle count. Eight times more physical time.

F_CPU is a compile-time agreement

```
#define F_CPU 16000000UL
#include <util/delay.h>

_delay_ms(500);
```

SOFTWARE ASSUMPTION

F_CPU = 16 MHz

PHYSICAL CONFIGURATION

CLKPR gives 2 MHz

The 500 ms delay becomes about 4,000 ms when software still assumes 16 MHz.

Writing CLKPR safely

```
static void clock_div_8(void)
{
    uint8_t saved_sreg = SREG;
    cli();
    CLKPR = _BV(CLKPCE);
    CLKPR = _BV(CLKPS1) | _BV(CLKPS0);
    SREG = saved_sreg;
}
```

1. **Preserve interrupt state**
2. **Open the four-cycle change window**
3. **Write the divisor bits**
4. **Restore interrupt state**

No fuse bits change during this demonstration.

Clock prescale

The clock model is now complete.

What changes when CLKPR divides the system clock by eight?

PHASE A

1 s blink cycle

PHASE B

8 s blink cycle

Observe, then explain the result with the clock model.

Demonstration setup and observation

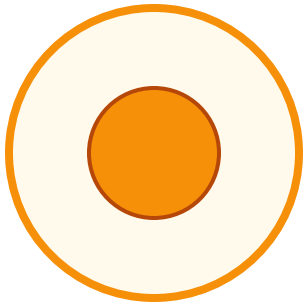
APPLY

Observe both phases. Then explain the factor of eight.

```
clock_div_1();
blink_cycles(4);

clock_div_8();
blink_cycles(4);
```

ONBOARD LED



PB5

Arduino D13

Only CLKPR changes at runtime. The source and F_CPU stay at 16 MHz.

The measured slowdown matches the clock model

Phase	Physical CPU clock	F_CPU assumption	Observed full blink
A	16 MHz	16 MHz	about 1 s
B	2 MHz after ÷8	still 16 MHz	about 8 s

The delay loop still contains the cycle count calculated for 16 MHz.

At 2 MHz, those cycles need eight times more physical time.

Clock errors propagate

Subsystem	What depends on clock	Typical symptom of a mismatch
software delay	cycle-counted loop	events occur too early or too late
timer and PWM	counter tick	wrong period, frequency, or duty timing
USART	baud-rate divisor	garbled or unreliable serial data
ADC	conversion clock	wrong speed or reduced accuracy
TWI/I²C	bit-rate generator	wrong bus frequency

Clock reasoning returns in Sessions 8, 10, 11, and 12.

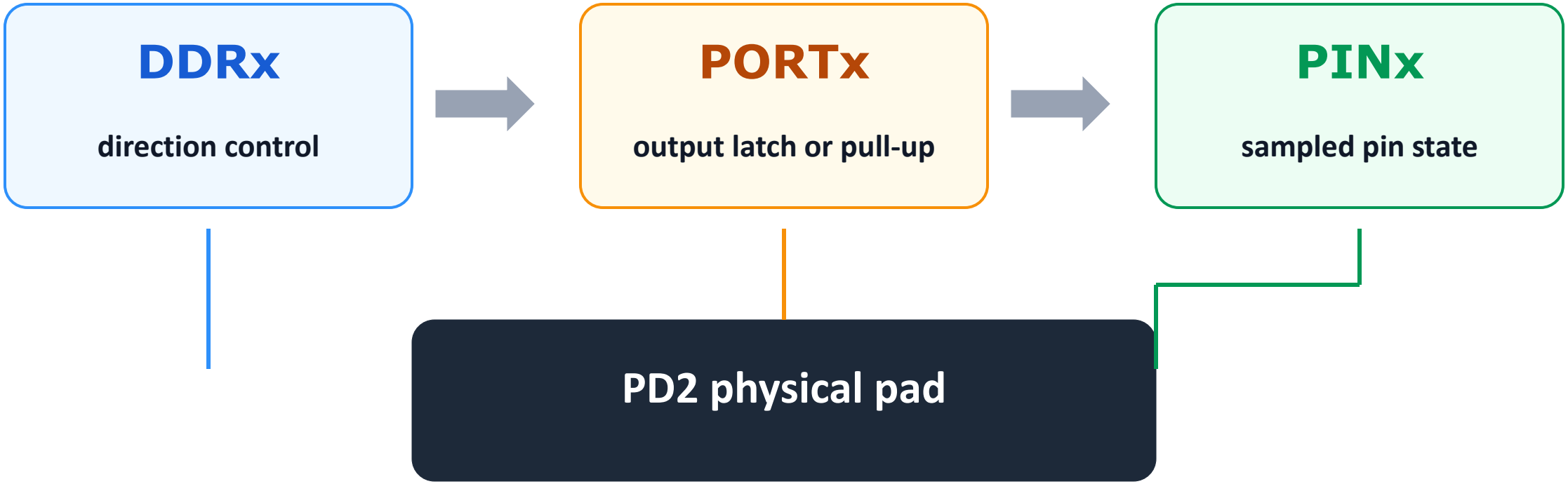
Arduino board ≠ Arduino programming

The Uno is a convenient ATmega328P development board.

Board label	ATmega328P pin	Hardware role today
D13	PB5	onboard LED
D2	PD2 / INT0	button input
D3	PD3 / INT1	challenge input
D7	PD7	external LED output

No Arduino.h · no pinMode · no digitalWrite · no attachInterrupt

Three GPIO registers, one pin



Remember the roles. The register names then become predictable.

DDRD2 and PORTD2 select four modes

DDRD2	PORTD2	Mode	PD2 behavior
0	0	input	high impedance; external circuit defines state
0	1	input + pull-up	weak internal path to VCC
1	0	output LOW	output driver sinks toward GND
1	1	output HIGH	output driver sources toward VCC

DDRx = direction · PORTx = output state or pull-up enable · PINx = actual pin state

Floating inputs have no default state

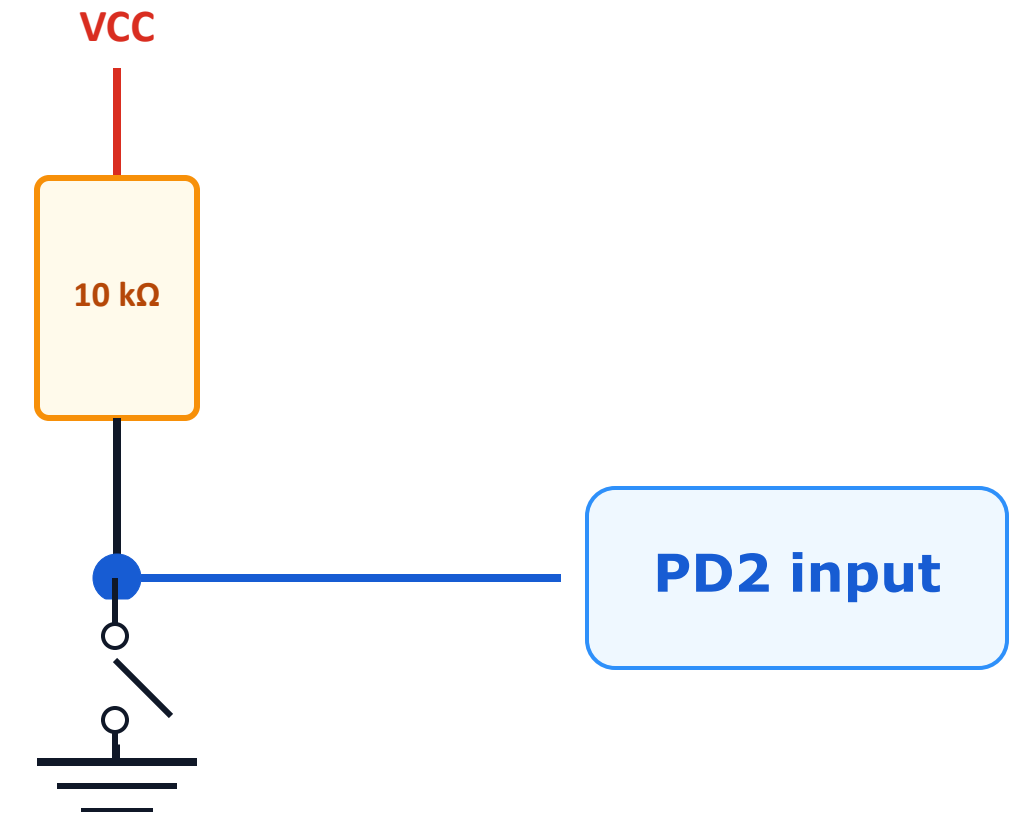


It can react to noise, leakage, nearby wiring, or your hand.

“Nothing connected” describes the circuit.
It does not define logic 0.

Check after explanation: should a floating reading stay stable?

An external pull-up defines a stable HIGH



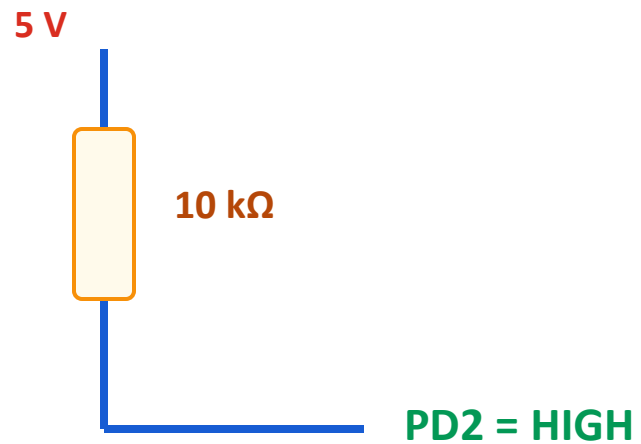
switch to GND

Switch	PD2 voltage	PIND2	Current
released	near VCC	1	almost none
pressed	near GND	0	limited by 10 kΩ

The resistor prevents a direct VCC-to-GND short when the switch closes.

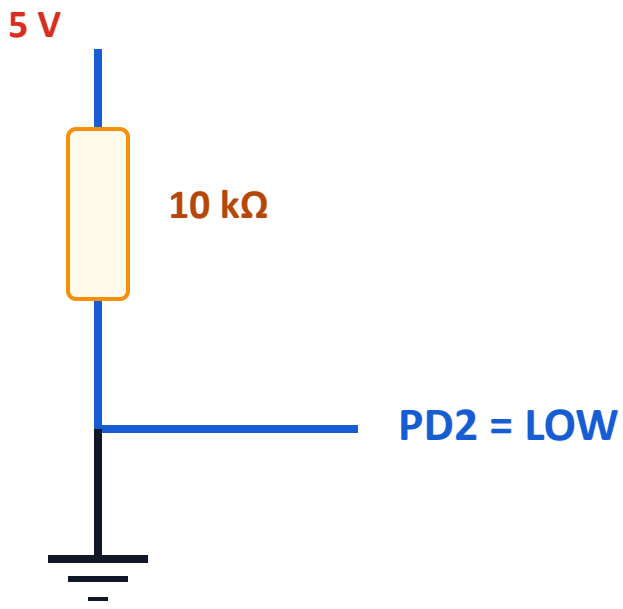
Pull-up current when the button is pressed

Released



Open switch means almost no DC path to GND.

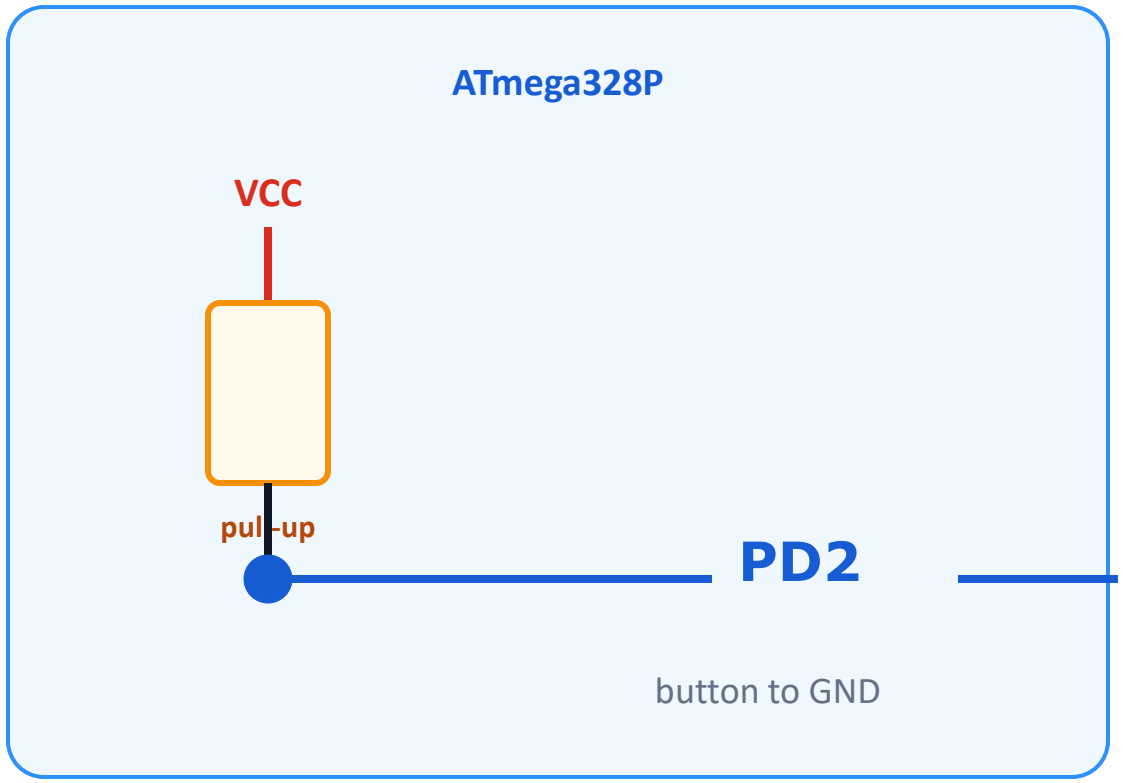
Pressed



I = V / R = 5 V / 10 kΩ ≈ 0.5 mA

The resistor limits current when the switch connects the node to GND.

The internal pull-up creates active-low input



DDRD2 = 0

PORTD2 = 1

released = 1

pressed = 0

A press produces a HIGH-to-LOW transition: a falling edge.

Pull-up comprehension check

Use the circuit model before looking at code.

1 **RELEASED BUTTON**

What does PIND2 read?

2 **PRESSED BUTTON**

What does PIND2 read?

3 **PRESS TRANSITION**

Which edge reaches INT0?

State all three answers, then justify them from the circuit.

Bare-metal button input

```
DDRD &= (uint8_t)~_BV(DDD2);
PORTD |= _BV(PORTD2);

if (!(PIND & _BV(PIND2)))
{
   /* button pressed */
}
```

1 **input direction**

2 **internal pull-up**

3 **read actual pin state**

The bitwise NOT clears only DDD2. The logical NOT tests active-low behavior.

Three bit operations behind the button code

```
DDRD &= (uint8_t)~_BV(DDD2);
```

CLEAR BIT 2

PD2 becomes an input

```
PORTD |= _BV(PORTD2);
```

SET BIT 2

enable the internal pull-up

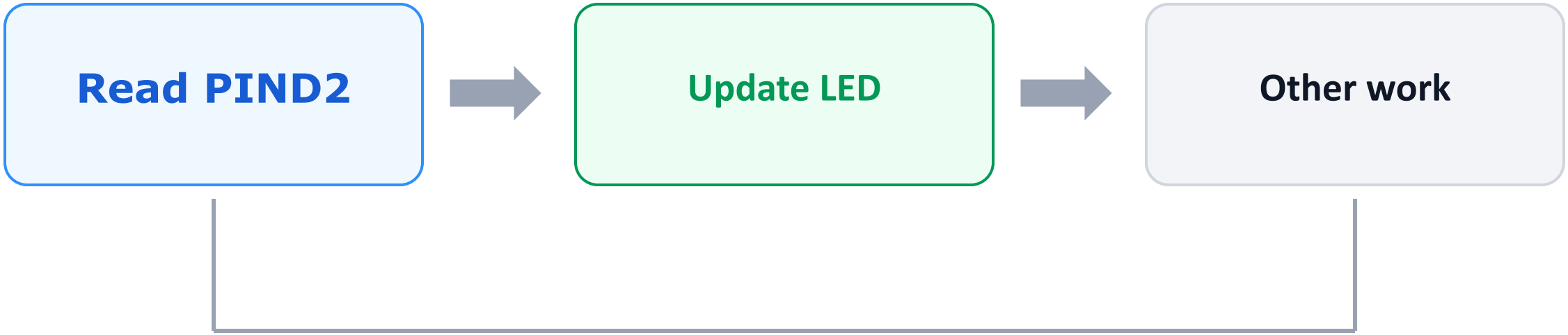
```
(PIND & _BV(PIND2)) == 0U
```

MASK AND COMPARE

true while the active-low button is pressed

The mask changes one selected bit while preserving the others.

Polling checks repeatedly




```
for (;;)
{
    if (!(PIND & _BV(PIND2)))
        PORTD |= _BV(PORTD7);
    else
        PORTD &= (uint8_t)~_BV(PORTD7);
}
```

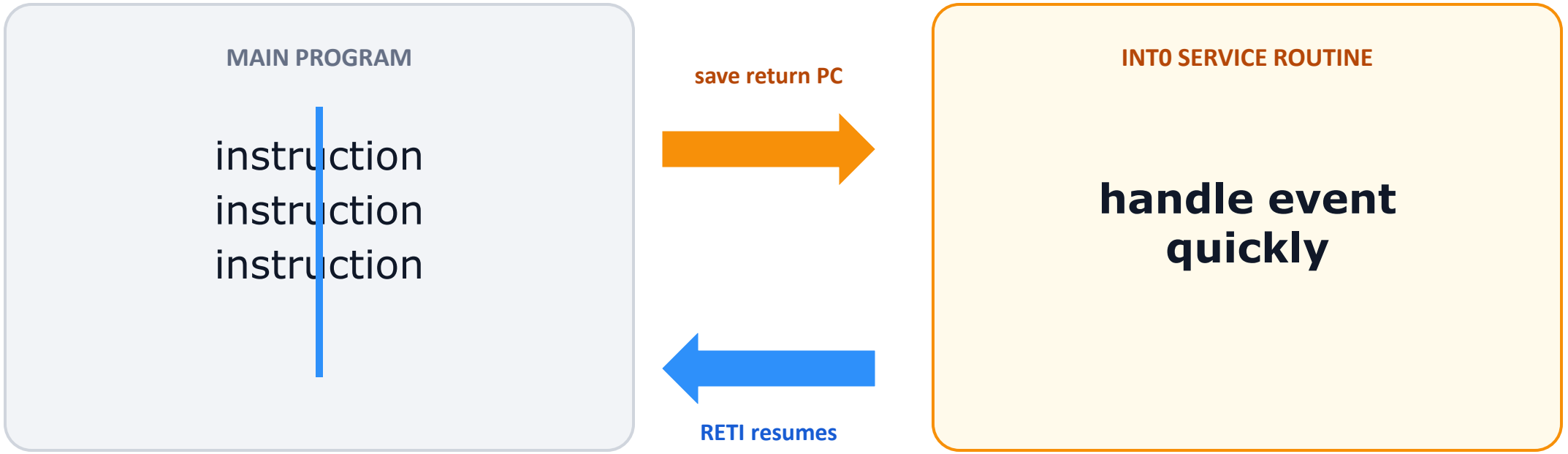
Polling can miss a short press



The event begins and ends while the CPU is not looking.

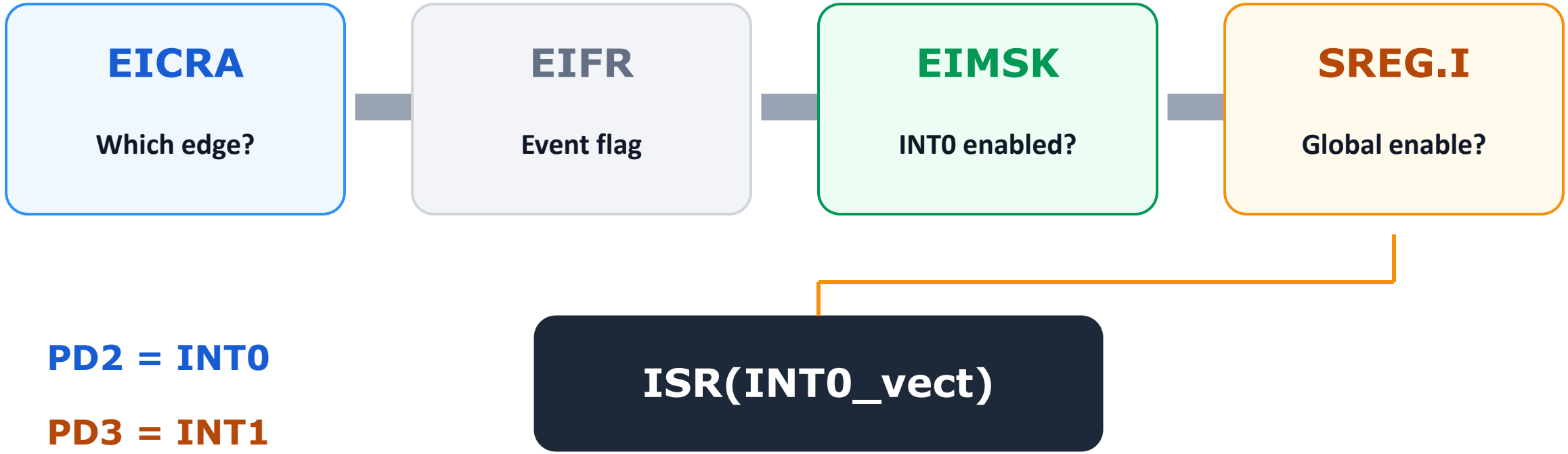
Live test: tap D2 quickly during the long delay.

Interrupts redirect execution



Hardware detects the event. The CPU finishes its current instruction, saves the return state, runs the ISR, then resumes.

INT0 passes four configuration gates



PD2 = INT0

PD3 = INT1

Configure INT0 for a falling edge

ISC01	ISC00	INT0 request condition
0	0	low level
0	1	any logical change
1	0	falling edge
1	1	rising edge

```
EICRA &= (uint8_t)~_BV(ISC00);
EICRA |= _BV(ISC01);
```

ISC01 = 1
ISC00 = 0

The button pull-up makes pressed equal a HIGH-to-LOW transition.

Minimal INT0 program

```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect)
{
    PORTD ^= _BV(PORTD7);
}

int main(void)
{
    DDRB |= _BV(DDB5);
    DDRD |= _BV(DDD7);
    DDRD &= (uint8_t)~_BV(DDD2);
    PORTD |= _BV(PORTD2);

    EICRA &= (uint8_t)~_BV(ISC00);
    EICRA |= _BV(ISC01);
    EIFR = _BV(INTF0);
    EIMSK |= _BV(INT0);
    sei();

    for (;;) { /* main work continues */ }
}
```

ISR practice

detect quickly

record quickly

leave quickly

Full runnable source adds a PB5 heartbeat to prove that main resumes.

Demo wiring: polling to interrupt

ARDUINO UNO AS ATmega328P BOARD

ATmega328P

PB5 / D13

PD2 / D2 / INT0

PD7 / D7

BUTTON

D2 ↔ GND

EXTERNAL LED

LED

330 Ω

D7 → resistor → LED anode
LED cathode → GND

D13 onboard LED shows main activity

All grounds must be common.

Demo 2 sequence

1	**Predict**	Will a quick press during the delay be detected?
2	**Polling**	Tap D2 briefly while PB5 is inside a 500 ms delay.
3	**Interrupt**	Upload INT0 version and repeat the same tap.
4	**Explain**	Hardware requested attention and main resumed afterward.

Expected: D7 responds during the main delay, while D13 continues its heartbeat after RETI.

Mechanical buttons create multiple edges

IDEAL BUTTON



one falling edge

MECHANICAL BUTTON



several falling and rising edges

The MCU may be correctly detecting several transitions from one physical press.

ISR habit: detect quickly, record quickly, leave quickly. Debounce returns in the keypad session.

Challenge: convert INT0 to INT1

Goal: move the button from D2 to D3 and preserve falling-edge behavior.

INT0 version	INT1 replacement
PD2 / D2	PD3 / D3
ISC01 = 1, ISC00 = 0	ISC11 = 1, ISC10 = 0
INTF0	INTF1
INT0	INT1
ISR(INT0_vect)	ISR(INT1_vect)

Keep DDRD3 = 0 and PORTD3 = 1 for the internal pull-up.

Lab Session 2 readiness

01	**REGISTER**	working state and memory-mapped hardware
02	**GPIO**	DDRx direction, PORTx output or pull-up, PINx sample
03	**BUTTON**	released HIGH, pressed LOW, falling edge
04	**INT0**	EICRA + EIFR + EIMSK + SREG.I + ISR
05	**ISR**	detect quickly, record quickly, leave quickly

The lab should no longer introduce a completely unfamiliar mechanism.

Exit ticket

1

CLOCK

F_CPU says 16 MHz, but CLKPR makes 2 MHz. A nominal 100 ms delay lasts about how long?

2

GPIO

DDRD2 = 0 and PORTD2 = 1. What mode is PD2 in?

3

INTERRUPT

Which two EICRA bits select a falling edge for INT1?

Answers: 800 ms · input with internal pull-up · ISC11 = 1 and ISC10 = 0

Five ideas to remember

01 Registers hold immediate CPU state and expose peripheral control.

02 Eight-bit describes the core data width, not every address or register.

03 Elapsed time equals instruction cycles divided by physical clock frequency.

04 DDRx selects direction, PORTx selects output or pull-up, and PINx samples the pad.

05 **A pull-up makes a grounded button active-low, and INT0 can respond without polling.**

NEXT: native AVR toolchain, build, upload, and debugging